

**DESIGN, DEVELOPMENT AND DEPLOYMENT OF DRIVER ASSISTANCE SYSTEM
USING OBJECT DETECTION MODEL ON MICROCONTROLLER SUPPORTED BY
NEURAL PROCESSING UNIT**

BITS ZG628T: Dissertation

by

Bhimesh Kiritkumar Bhatt BITS ID: 2024HT01120

Dissertation work carried out at

Tata Consultancy Services, Mumbai

Submitted in partial fulfilment of **M. Tech. Embedded Systems** degree programme

Under the Supervision of **(Insert Supervisor Name)**



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE PILANI (RAJASTHAN)

March, 2026

ABSTRACT

Road safety in India remains a critical challenge, with accidents often caused by human error, delayed reactions, and poor situational awareness. Unlike structured Western traffic environments, Indian roads present heterogeneous conditions—ranging from auto-rickshaws and tractor-trolleys to pedestrians, stray animals, and inconsistent lane markings. This dissertation addresses the urgent need for an affordable, real-time driver assistance solution tailored to such unstructured traffic.

The project proposes a proof-of-concept (PoC) driver assistance system leveraging Neural Processing Units (NPUs) integrated into microcontrollers for edge AI deployment. Unlike cloud-based approaches, edge inference ensures low latency, reduced bandwidth usage, and improved privacy. The selected hardware platform, STMicroelectronics STM32N6, features the Neural-ART accelerator delivering 600 GOPS, enabling efficient execution of lightweight convolutional neural networks (CNNs).

A comparative literature review highlights the evolution of NPUs, their architectures (systolic arrays, RISC-V extensions, precision-scalable designs), and their suitability for embedded AI. Object detection algorithms are analyzed, with emphasis on one-stage detectors such as YOLO variants, which balance accuracy and speed for real-time deployment. Custom training is performed using localized datasets (e.g., Indian Driving Dataset), ensuring robustness against diverse traffic scenarios.

The methodology involves model optimization through quantization and pruning, deployment on the STM32N6 MCU, and evaluation against key performance objectives: 30+ FPS inference speed, high detection accuracy, compact hardware footprint, and cost-effectiveness. Preliminary experiments with YOLO models demonstrate promising results, though challenges remain in thermal management, memory fragmentation, and balancing power-performance trade-offs.

This work contributes toward scalable, low-cost driver assistance technologies for Indian roads, bridging the “edge-reality gap” between advanced AI models and constrained embedded hardware. The broader vision is to enable safer mobility through localized, efficient, and practical AI solutions.

Signature of the Student

Name: Bhimesh Bhatt

Date: March 21, 2026

Place: Mumbai

Signature of the Supervisor

Name: Apoorve Aggarwal

Date: March,21 2026

Place: New Delhi

Contents

1. Introduction.....	4
2. The Research Gap.....	5
3. Literature Review.....	7
3.1. Neural Processing Units (NPUs).....	7
3.2. Object Detection Algorithms.....	14
3.3. Datasets for Training.....	19
3.4. Summary of Literature Review.....	21
4. Methodology & Experimental Setup.....	22
4.1. Methodology.....	22
4.2. Experimental Setup.....	22
5. Model Training on custom dataset.....	24
6. Implementation & Analysis (Work in progress).....	31
7. Results and Discussion (Work in progress).....	31
8. Conclusion & Future Work (Work in progress).....	31
9. References & Appendices (Work in progress).....	31

Figures

Figure 1 - Hardware Accelerator Landscape.....	7
Figure 2 - NPU at the sweet spot for AI at Edge.....	8
Figure 3 - NPU Systolic Array.....	9
Figure 4 - Generic CNN Architecture.....	14
Figure 5 - Yolo11n Performance Score.....	26
Figure 6 - Yolo11n Performance Charts.....	26
Figure 7 - Yolo11n Confusion Matrix.....	27
Figure 8 - Yolo11n Test Result.....	27
Figure 9 - Yolo26n Performance Score.....	28
Figure 10 - Yolo26n Performance Charts.....	28
Figure 11 - Yolo26n Confusion Matrix.....	29
Figure 12 - Yolo26n Test Result.....	29

Tables

Table 1 - 2026 Edge AI MCUs/Processors.....	10
Table 2 - CNN Models Comparison.....	15
Table 3 - Latest CNN Models and Project Suitability.....	16

1. Introduction

Road safety remains one of the most pressing challenges in India, where thousands of lives are lost annually due to road accidents. According to national statistics, a significant proportion of these accidents are attributed to human error, delayed reaction times, and inadequate situational awareness. With the rapid increase in vehicular density and diverse traffic conditions—ranging from heavy trucks and buses to two-wheelers, pedestrians, and stray animals—the need for intelligent driver assistance systems has become critical. Developing an effective Advanced Driver Assistance System (ADAS) for this region is uniquely challenging due to **"Unstructured Traffic."** Unlike Western datasets (e.g., KITTI), Indian roads feature a heterogeneous mix of vehicles (auto-rickshaws, tractor-trolleys), pedestrians, stray animals, and a lack of consistent lane markings or signage. For a safety system to be viable, it must detect these anomalies with near-zero latency to provide real-time alerts. A cost-effective, reliable, and compact solution that can alert drivers to potential hazards has the potential to significantly reduce accidents and save lives.

The goal of this dissertation project is to design and implement a proof-of-concept (PoC) driver assistance solution that leverages modern advances in **Neural Processing Units (NPUs)** and **edge computing**. Unlike traditional cloud-based systems, edge devices enable real-time inference directly on the vehicle, reducing latency and ensuring timely alerts. The proposed solution will focus on detecting dangerous conditions such as vehicles ahead, pedestrians, animals on the road, and warning signs, thereby enhancing driver awareness in complex traffic environments.

The Core Conflict: The "Edge-Reality" Gap

To implement such a system in a real-world vehicle, the processing must happen at the "Far Edge"—on a microcontroller (MCU) or a low-power SoC—to ensure the system is cost-effective, compact, and functions without cloud dependency. However, a major conflict exists between the software requirements and hardware constraints:

- **Model Complexity vs. Real-Time Constraints:** State-of-the-art object detection models (like YOLOv11) are computationally heavy. Implementing them on an MCU requires aggressive optimization (Quantization and Pruning) which often risks missing small, critical objects like a distant pedestrian or a pothole.
- **The Power-Performance Paradox:** To achieve the necessary Frame Per Second (FPS) for high-speed driving alerts, hardware must run at high clock speeds, leading to thermal issues and high power draw. Conversely, low-power MCUs often lack the NPU (Neural Processing Unit) throughput required to process high-resolution video streams (720p or 1080p).
- **Domain Gap in Literature:** Most existing literature focuses on standard datasets. There is a critical lack of research on how specialized NPU architectures (like the ARM Ethos-U or RISC-V accelerators) handle the specific feature maps generated when trained on chaotic, high-density Indian traffic data.

2. The Research Gap

While researchers have explored lightweight CNNs (MobileNet, SqueezeNet) and various hardware (STM32, NXP i.MX), there is no clear "Gold Standard" for a **Drive Assistance System** that utilizes custom-trained models on Indian datasets for MCU-class deployment. Most current solutions are either too expensive (using high-end NVIDIA GPUs) or too inaccurate for reliable safety alerts.

Motivation and Objectives

The motivation for this project arises from two converging trends:

1. **Advances in NPUs and microcontrollers:** NPUs are increasingly being integrated into microcontrollers and edge devices, enabling efficient execution of deep learning models with low power consumption.
2. **Progress in object detection algorithms:** State-of-the-art algorithms such as YOLO, SSD, and EfficientDet have demonstrated high accuracy and speed, making them suitable candidates for real-time deployment in constrained environments.

The primary objectives of this dissertation are:

- To conduct a **high-level literature review** of NPUs, their architectures, and their suitability for edge deployment.
- To study **microcontrollers with integrated NPUs**, comparing their performance, cost, and scalability, and selecting the most appropriate platform for implementation.
- To review **object detection algorithms** with a focus on lightweight models optimized for edge devices, and select the most suitable algorithm for this application.
- **Literature-Driven Selection:** Identifying the optimal pairing of a lightweight CNN and an NPU-equipped Microcontroller.
- **Custom Optimization:** Training the model using the **Indian Driving Dataset (IDD)** or **DriveIndia** to ensure high mAP (Mean Average Precision) for local road conditions.
- **Hardware Implementation:** Deploying the model onto the selected MCU, focusing on the trade-off between **Power Consumption (Watts)**, **Inference Latency (ms)**, and **Detection Accuracy**.
- Propose optimization techniques, model scheduling improvement, or quantization method
- Discuss the challenges related to Thermal constraints, Power gating. Memory fragmentation
- To implement the selected object detection model on the chosen microcontroller and evaluate its performance against defined metrics.

Performance Objectives

The success of the proposed solution will be measured against four key performance objectives:

- **Performance:** The system should achieve real-time inference at a minimum of 30 frames per second (fps).
- **Accuracy:** False positives must be minimized to avoid unnecessary driver distractions.
- **Size:** The hardware should be compact enough to be mounted conveniently on a car dashboard.
- **Cost:** The device must be affordable to ensure accessibility for a wide range of users.

Limitation of this project

This dissertation will not attempt to build a fully commercial driver assistance system but rather a **proof-of-concept prototype** that demonstrates feasibility. The scope includes:

- Reviewing the **current state of NPUs** and their integration into microcontrollers.
- Evaluating **object detection algorithms** for edge deployment.
- Designing and training a model using a relevant dataset.
- Implementing and testing the solution on an edge device.

The broader vision is to contribute towards affordable and scalable driver assistance technologies that can be adapted to Indian traffic conditions, thereby improving road safety and reducing fatalities.

3. Literature Review

3.1. Neural Processing Units (NPU)

This literature review provides a detailed examination of Neural Processing Units (NPUs) covering their necessity at the edge, their architectural underpinnings, historical evolution, and the current state of the market.

Neural networks have become a dominant computational workload across cloud and edge platforms, but their rapid growth in model size and deployment diversity has exposed hardware bottlenecks that are increasingly dominated by memory movement, communication, and irregular operators rather than peak arithmetic throughput. There is a large continuum of current technology landscape for hardware acceleration of deep learning, spanning Graphics Processing Units (GPUs) and tensor-core architectures, domain-specific accelerators (e.g., Tensor Processing Units (TPUs)/Neural Processing Units (NPUs)), Field-Programmable Gate Array (FPGA)-based designs, Application-Specific Integrated Circuit (ASIC) inference engines, and emerging Large Language Model (LLM)-serving accelerators such as Language Processing Units (LPUs).

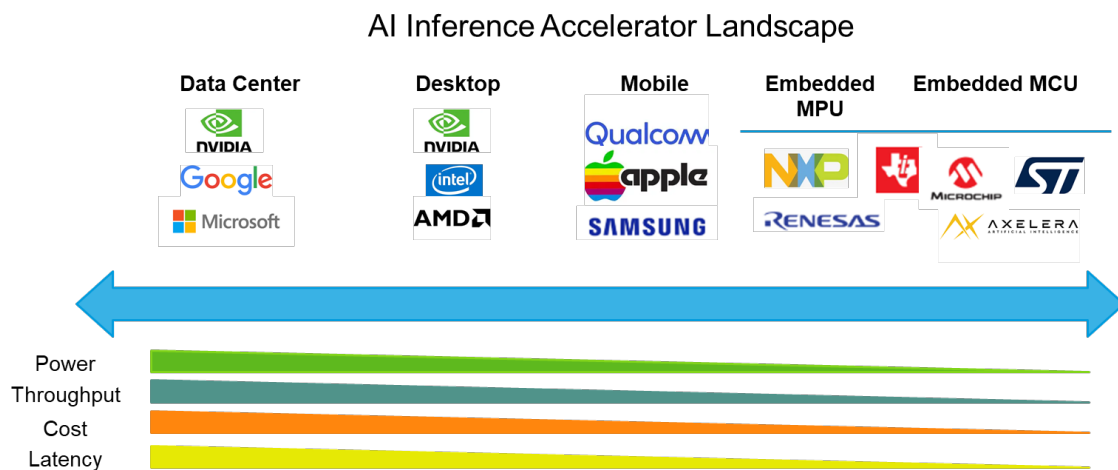


Figure 1 - Hardware Accelerator Landscape

While there is plethora of AI Accelerations technologies exist today, Energy Consumption, Throughput / Latency and Cost play a vital role in using these technologies. Designing and deploying neural-network accelerators requires navigating a multi-objective optimization problem where improvements along one axis often degrade another. Power consumption is a first-order constraint because it determines thermal limits, battery lifetime, and operating cost. Throughput and latency requirements impose different architectural and scheduling pressures. Datacentre inference serving may prioritize tokens-per-second at high utilization while still meeting strict end-to-end latency targets, whereas mission critical and data privacy applications care about minimum latency and processing near sensor. In line with our selected application of Driver Assistance System, latency is of most critical performance parameter followed by accuracy and power.

Need for Hardware Acceleration at the Edge

The demand for deep learning (DL) on edge devices—such as smartphones, IoT sensors, and autonomous vehicles—is driven by several critical factors:

Latency and Real-time Response: Cloud-based inference incurs significant latency due to data transfers between the device and remote servers. Edge acceleration is vital for applications requiring immediate action, such as autonomous driving.

Power and Energy Efficiency: Edge devices are typically battery-powered and resource-constrained. Moving data to the cloud is energy-expensive; therefore, performing computation locally reduces the power overhead.

Privacy and Security: Processing sensitive data (e.g., medical or security images) on-device avoids the risks associated with transmitting data over networks.

Bandwidth Constraints: The massive volume of data generated at the edge often makes it impractical to send everything to the cloud for processing.

Figure below shows, that NPU falls on a sweet spot on a continuum of High End Microprocessors on one end and Specialised ASICS on other end in terms of Flexibility, Latency and Cost.

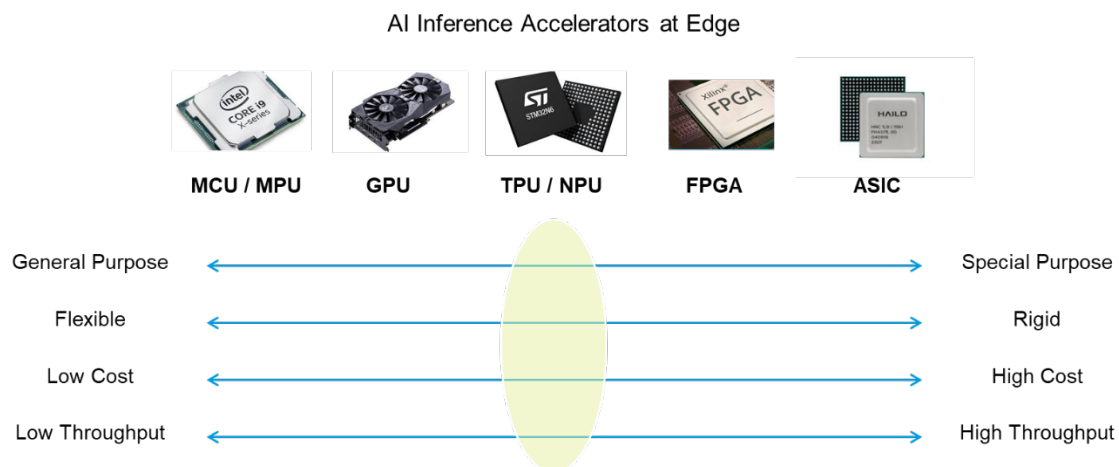


Figure 2 - NPU at the sweet spot for AI at Edge

What is an NPU and How it Helps in AI Acceleration

An NPU (Neural Processing Unit) is a specialized processor custom-designed to accelerate machine learning models. Unlike a CPU, which handles sequential control, or a GPU, which excels at streaming parallel graphics data, an NPU is built from the ground up for the core math of AI workloads: scalar, vector, and tensor operations.

Key ways NPUs accelerate AI:

Optimization for Neural Math: They are designed specifically for operations like convolutions and matrix multiplications, which make up the bulk of neural network layers.

High Compute Utilization: They focus on maximizing compute throughput while minimizing the energy cost of moving data between memory and processing elements.

Low Precision Efficiency: NPUs are often designed for high volumes of low-precision computation (e.g., 4-bit or 8-bit integers), which offers higher performance-per-watt than the high-precision floating-point math used by traditional CPUs.

Prevalent NPU Architectures

The development of NPUs has evolved through several distinct phases:

Historical Evolution: The field originated in the late 1980s with early CNNs like LeNet, followed by a period of stagnation in the early 2000s due to hardware limitations and a lack of data. The revival began around 2006 with the use of GPUs and the establishment of large datasets like ImageNet. NPU designs vary based on their target application (e.g., cloud vs. tiny IoT), but several architectures are prevalent:

Systolic Arrays: The most famous architecture (used in Google's TPU), which consists of a 2D grid of Processing Elements (PEs) that pass intermediate results directly to their neighbours to exploit parallelism and pipelining.

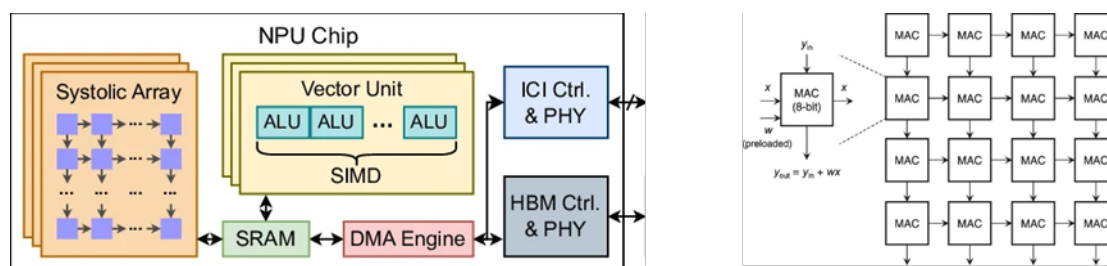


Figure 3 - NPU Systolic Array

In or Near-Memory Compute: Commercial NPUs often favor this design, utilizing small, tightly coupled on-chip memories and integrated data-mover engines to execute workloads that reside in off-chip memory efficiently.

Hardware-Aware Neural Architecture Search (HW-NAS): Significant research has focused on automating the design of neural networks to fit specific hardware constraints, optimizing for a balance between accuracy and latency.

Model Compression Techniques: To make models fit on NPUs, efforts have been made in quantization (reducing bit-width), pruning (removing redundant weights), and parsing.

Precision-Scalable Architectures: These units can be configured at runtime to handle different bit-widths (e.g., 4, 8, or 16-bit). They utilize two main strategies:

Divide-and-Conquer (D&C): Bottom-up designs where the logic is formed by combining several identical sub-blocks (e.g., BitFusion, BitBlade).

Subword-Parallel (SWP): Top-down designs where a full-precision multiplier is gated to reuse arithmetic cells in reduced-precision modes.

RISC-V-Based NPUs: Emerging architectures leverage the open RISC-V ISA to balance scalar, vector, and matrix capabilities on a single platform, often using custom extensions like the Attached Matrix Extension (AME) for specialized acceleration. Organizations like RISC-V International are working to standardize matrix and vector operations at the ISA level to ensure software portability across different AI accelerators.

Current State of the Area

Focus on HW/SW Co-Design: Modern state-of-the-art performance is determined by the tight coupling between the compiler and hardware memory hierarchies rather than just raw "Peak TOPS" (Tera Operations Per Second).

On-Device Generative AI: There is a major shift toward enabling Generative AI (e.g., Large Language Models) to run locally on handheld and mobile devices, requiring refreshed heterogeneous architectures.

TinyML and IoT Integration: NPUs are now being integrated into microcontrollers for "always-on" low-power edge AI applications.

Fragmentation vs. Standardization: The hardware landscape is currently fragmented, but there is strong momentum toward using the open RISC-V standard as a baseline for AI accelerators in automotive and data centre sectors.

Latest NPU Chips and Manufacturers

From the literature review it is understood that there is a clear shift from CPU-only inference to dedicated hardware acceleration. The landscape has changed significantly in 2026, with NPUs (Neural Processing Units) moving from high-end SoCs down into low-cost, real-time microcontrollers.

Below is a list of major manufacturers offering integrated NPUs/AI accelerators in their MCUs and application processors.

Manufacturer	Chip Family	NPU/Accelerator	Peak AI Performance	Target Application
ST	STM32N6	Neural-ART	600 GOPS	Computer Vision/ISP
TI	AM13Ex	TinyEngine	2.56 GOPS	Motor Control/Robotics
NXP	MCX N94x	eIQ Neutron	~150-200 GOPS	Industrial IoT/Secure AI
Renesas	RZ/V2H	DRP-AI	High (Vision-optimized)	4K Vision/Surveillance
Ambiq	Apollo510	Cortex-M55 + Helium	10x Latency gain	Ultra-low power Wearables

Table 1 - 2026 Edge AI MCUs/Processors

Selection of STM32N6 for experiment

The STM32N6 series is STMicroelectronics' first high-performance microcontroller (MCU) family to feature a dedicated integrated **Neural Processing Unit (NPU)**, designed specifically for efficient **Edge AI** and machine learning tasks. It brings microprocessor-class performance to the MCU level, allowing complex neural networks to run locally with low power consumption and high efficiency.

Architecture and Neural-ART Accelerator

The STM32N6 architecture marks a significant paradigm shift from the classic Von Neumann architecture to a flexible, dedicated dataflow stream processing engine. Key architectural components include:

Core: It is based on the **Arm Cortex-M55 core** running at 800 MHz. This is the first CPU in its class to include **Arm Helium technology** (M-Profile Vector Extension), which significantly boosts signal processing (DSP) and machine learning performance.

Neural-ART Accelerator™ (NPU): This in-house developed NPU is clocked at 1 GHz and provides **600 GOPS** (Giga-Operations Per Second) of computing power with an efficiency of **3 TOPS/W**. It enables real-time neural network inference for tasks such as computer vision and audio analysis.

Memory Hierarchy: The series offers **4.2 MB of contiguous embedded SRAM**, the largest in the STM32 family, which is critical for handling large activation maps in neural networks and high-resolution graphics. It also supports high-speed external memory via hexa-SPI, OCTOSPI, and FMC interfaces.

Multimedia and Vision: It includes a dedicated computer vision pipeline with a MIPI CSI-2 interface, an integrated Image Signal Processor (ISP), a hardware H.264 encoder, and the NeoChrom™ Accelerator for advanced 2.5D graphics.

Security: The architecture is highly secure, featuring TrustZone-M and target certifications for SESIP Level 3 and PSA Level 3.

Use Cases

The STM32N6 is designed for a wide array of intelligent applications across various sectors:

Smart Industry: Industrial Predictive Maintenance Gateways that collect high-frequency vibration, temperature, and acoustic data to detect early faults using the NPU.

Smart Cities and Homes: Sound recognition, object classification, and face/people detection for automated home systems and urban monitoring.

Automotive: Advanced Driver Assistance Systems (ADAS) and subject tracking in smart sensors.

Medical and Healthcare: Real-time data analysis for medical monitoring devices.

Multimedia Products: Feature-rich wearable technology, video recording, and streaming over high-speed interfaces like Gigabit Ethernet with TSN support.

Advantages

High Performance at Low Power: It can handle 75x more frames than an STM32H7 and infers 25x faster than an STM32MP1 microprocessor, yet it does not require passive cooling even in high-temperature industrial environments.

Reduced Bandwidth and Latency: By processing data locally (at the edge), devices only transmit insights rather than raw data streams, reducing bandwidth usage by over 90% and eliminating dependency on the cloud.

Real-time Responsiveness: The dedicated NPU offloads AI tasks from the main CPU, allowing the Cortex-M55 to remain responsive for other system tasks, such as maintaining a smooth display.

Ecosystem Integration: It is fully supported by the STM32Cube.AI ecosystem, which automatically maps neural network layers to the NPU to maximize efficiency.

Resiliency: Devices can operate offline and only share insights when connectivity is available, making them ideal for mission-critical IoT.

The STM32N6 family includes versions like the STM32N6x7 (AI line) with the NPU and the STM32N6x5 (GP line), a general-purpose version without the NPU but with the same high-performance core and graphics capabilities.

The STM32N6x7 series (the AI-specific line of the STM32N6 family) is a high-performance solution for object detection at the edge due to its dedicated hardware acceleration, vision-specific peripherals, and extreme power efficiency.

Key reasons why it is a good fit include:

Dedicated AI Performance and Throughput

Neural-ART Accelerator™ (NPU): This in-house NPU provides 600 GOPS of computing power, enabling real-time neural network inference. It utilizes a dataflow stream processing engine that reduces memory load and power consumption by shifting away from classic Von Neumann architecture.

Significant Benchmarking Gains: In demonstrations, the STM32N6 handled 75x more frames than the STM32H7 while running a YOLO derivative neural network for detecting and locating multiple people. It also provides massive performance uplifts for common object detection models, such as 127x for TinyYolo v2 and 59x for Yolo v8n compared to a standard Cortex-M55 core.

Vector Processing: Its Arm Cortex-M55 core features Helium technology (M-Profile Vector Extension), which significantly boosts machine learning and signal processing performance compared to older MCU generations.

Integrated Vision Pipeline

Real-Time Image Processing: The series features a dedicated computer vision pipeline with a MIPI CSI-2 interface and an integrated Image Signal Processor (ISP). This ISP can process three parallel pipelines from the same input stream, enabling sophisticated real-time analysis.

Large Embedded Memory: It includes 4.2 MB of contiguous embedded SRAM, the largest in the STM32 family. This large memory pool is critical for handling the high-resolution activation maps required by object detection models.

Efficiency and System Responsiveness

CPU Offloading: By redirecting AI compute tasks to the NPU and camera preview functions to the vision pipeline, the main Cortex-M55 core remains free to handle other system tasks. This results in a smooth and responsive user interface that can follow fast-moving subjects without lag.

Low Power/Thermal Profile: Despite its performance, the STM32N6 operates with an efficiency of 3 TOPS/W and does not require passive cooling or heat sinks, even in high-temperature industrial environments.

Secure Edge Operations

High-Level Security: For mission-critical IoT applications, the series provides SESIP Level 3 and PSA Level 3 target certifications. It uses TrustZone-M and a Memory Protection Unit (MPU) to isolate sensitive AI models and data from cyber threats.

Local Intelligence: Devices can process video data locally and only transmit insights (e.g., object counts or fault alerts) rather than raw streams, reducing bandwidth usage by over 90% and improving privacy.

3.2. Object Detection Algorithms

In this literature review we compare and analyze the evolution of Convolutional Neural Networks (CNNs) and the specific technical advantages of **YOLO11** and **YOLO26** for edge deployment on microcontrollers.

Introduction to AI/ML and Object Detection

Artificial Intelligence (AI) and Machine Learning (ML) have transformed computer vision by enabling machines to interpret and understand visual data. At the core of these technologies are deep learning models, particularly Convolutional Neural Networks (CNNs), which excel at extracting hierarchical features from images. Object detection is one of the most impactful applications of AI/ML, combining image classification and localization to identify and locate multiple objects within a scene.

In the context of driver assistance systems, object detection plays a crucial role in recognizing vehicles, pedestrians, animals, and road signs in real time. By providing timely alerts, such systems can enhance driver awareness and reduce accidents. The challenge lies in deploying these models on **edge devices** with limited computational resources, where efficiency, speed, and accuracy must be balanced.

Core Architecture and Functionality

A CNN is a specialized type of neural network algorithm designed to recognize patterns in image data. Unlike general neural networks, CNNs utilize **convolutional layers** which use learned kernels (weights) to extract local features, such as edges and textures, that distinguish different images. The fundamental building blocks of a CNN include:

Tensors: n-dimensional matrices (typically 3D for images) that hold data.

Convolutional Layers: These perform elementwise dot products between a kernel and the input, producing activation maps.

Activation Functions: ReLU (Rectified Linear Unit) is widely used to introduce non-linearity, allowing the network to learn complex patterns and preventing deep architectures from devolving into single linear layers.

Pooling Layers: These reduce the spatial extent of the network (e.g., Max-Pooling), which decreases the number of parameters and computational load while helping avoid overfitting.

Flatten and Softmax Layers: The flatten layer converts 3D tensors into 1D vectors for final classification, while Softmax scales outputs into probabilities that sum to one.

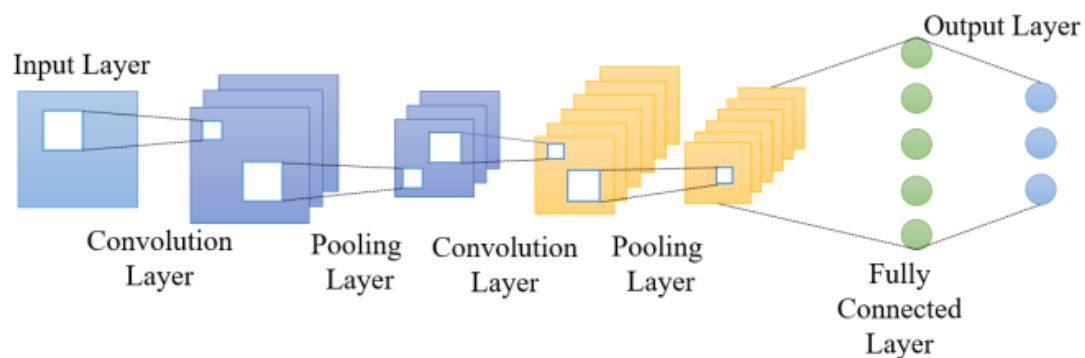


Figure 4 - Generic CNN Architecture

Evolution of Object Detection

Object detection has progressed from traditional handcrafted features to deep learning paradigms, primarily categorized into two frameworks:

- 1. Two-Stage Detectors:** Models like the Faster R-CNN series use a Region Proposal Network (RPN) to identify candidate regions before refining them in a second stage. These are noted for high accuracy and recall but are often slower due to the two-step refinement process.
- 2. One-Stage Detectors:** Models such as YOLO (You Only Look Once) and SSD perform classification and bounding box regression in a single forward pass. These architectures prioritize inference speed and reduced complexity, making them better suited for real-time applications.

Comparison of CNN Models

Over the past decade, several object detection algorithms have been developed, each offering trade-offs between accuracy, speed, and resource requirements. Recent benchmarks demonstrate clear trade-offs between accuracy (mAP) and computational efficiency (FPS, parameters):

Model Class	Representative Models	Strengths	Weaknesses
Two-Stage	Faster R-CNN	High accuracy and recall due to multi-step refinement.	High latency (~7 FPS) and large parameter counts (~136M).
One-Stage	YOLOv5, YOLOv8	High inference speeds (100+ FPS) and competitive accuracy for real-time use.	Can struggle with extreme scale variations or dense occlusions.
Transformer-Based	DETR, RT-DETR	Excels at global context modelling and long-range dependencies; eliminates manual components like NMS.	Historically high computational overhead, though RT-DETR achieves 108 FPS with 53.1% mAP.
Edge-Optimized	YOLO11, YOLO26	Extreme efficiency, "edge-first" design, and NMS-free inference.	Lower absolute mAP compared to much larger, server-grade models.

Table 2 - CNN Models Comparison

Some more latest Object Detection Model comparisons from a project suitability perspective.

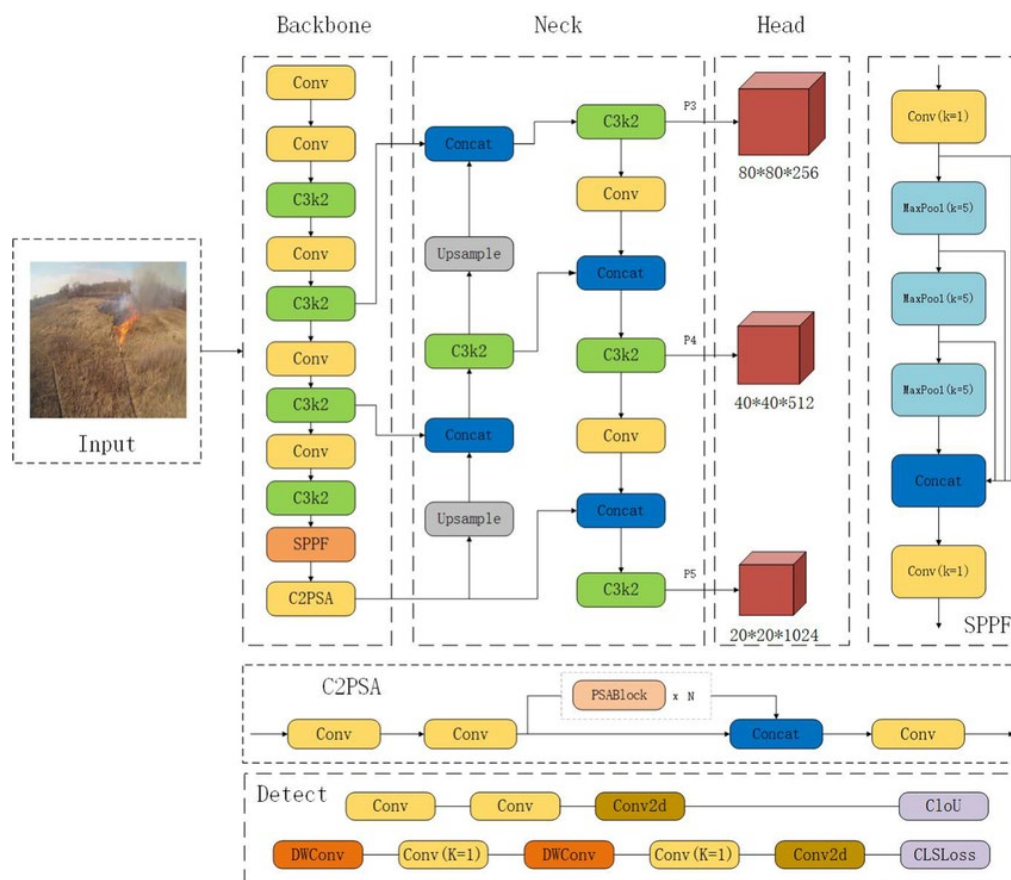
Model	Description	Strength	Limitation	Suitability
YOLO (You Only Look Once)	YOLO revolutionized object detection by framing it as a regression problem, predicting bounding boxes and class probabilities directly from images in a single pass.	Extremely fast, capable of real-time detection (>30 fps).	YOLOv3, YOLOv5 and YOLOv8 introduced lightweight versions optimized for edge devices. YOLO11 and YOLO26 (latest) continues this trend with enhanced accuracy and efficiency.	Ideal for driver assistance systems where speed and responsiveness are critical.
SSD (Single Shot Detector)	SSD predicts bounding boxes and class scores in a single step using feature maps at multiple scales.	Good balance between accuracy and speed; simpler than YOLO in design.	Slightly less accurate for small object detection compared to YOLO.	Effective for general-purpose detection, but may struggle with small or distant objects common in road scenarios.
EfficientDet	EfficientDet builds on EfficientNet backbones and introduces compound scaling to balance accuracy and efficiency.	High accuracy with relatively small model size; scalable across different resource budgets.	More complex training and tuning; inference speed may be lower than YOLO on constrained hardware.	Useful when accuracy is prioritized over speed, though optimizations are needed for real-time edge deployment.
MobileNet-based Detectors	MobileNet is a lightweight CNN designed for mobile and embedded devices, often used as a backbone for detection models.	Extremely small footprint, low power consumption, efficient on NPUs and microcontrollers .	Extremely small footprint, low power consumption, efficient on NPUs and microcontrollers .	Best for ultra-low-power devices where cost and size are critical, but may require trade-offs in detection accuracy.

Table 3 - Latest CNN Models and Project Suitability

Microcontroller Units (MCUs) are highly resource-constrained, typically possessing very limited RAM (often <1MB) and Flash memory (often <2MB). Deploying standard CNNs on these platforms is challenging because model weights and intermediate feature maps often exceed these hard memory limits.

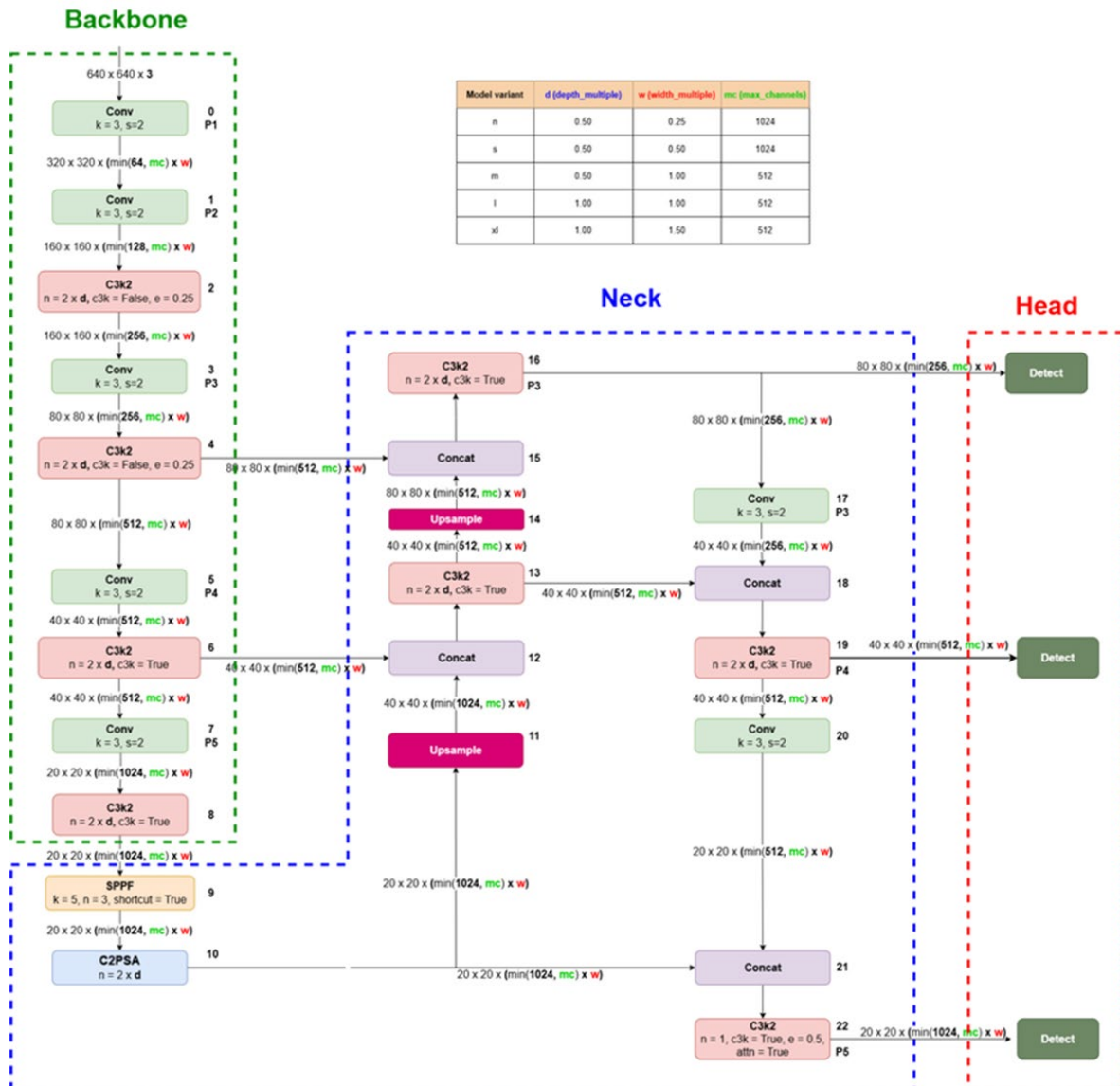
For this project we are selecting Detect and within it Nano model.

- **Refined Bottlenecks:** YOLO11 introduces the C3k2 block, a module that optimizes gradient flow and improves parameter efficiency, allowing the model to remain accurate while staying small.
- **Spatial Attention:** It incorporates an enhanced spatial attention mechanism (C2PSA), which helps the model focus on important regions, critical for identifying small objects in edge scenarios.
- **Versatility:** It supports a wide array of tasks (detection, segmentation, pose estimation) within a unified framework that is easier to deploy on devices like the MCU.



Why YOLO26n (detect) is the Premier Fit for MCUs:

- **DFL Removal:** YOLO26 eliminates Distribution Focal Loss (DFL). DFL requires complex operations often unsupported by low-power edge accelerators; its removal results in up to 43% faster CPU inference.
- **End-to-End NMS-Free Design:** By natively predicting exactly one box per object, YOLO26 eliminates the need for Non-Maximum Suppression (NMS) post-processing. This removes a significant latency bottleneck and ensures deterministic inference times, which is critical for robotics and autonomous systems.
- **Small Object Detection:** Through ProgLoss (Progressive Loss Balancing) and STAL (Small-Target-Aware Label Assignment), YOLO26 achieves superior precision on tiny objects, essential for IoT sensor analysis and aerial drone imagery.
- **Optimized Memory Footprint:** The YOLO26-nano (YOLO26n) variant achieves approximately 40.9% mAP with only 2.4M parameters and 5.4B FLOPs, significantly outperforming predecessors in the performance-to-size ratio required for microcontrollers.



3.3. Datasets for Training

A critical component of this project is the selection and preparation of datasets for training object detection models. The choice of dataset directly influences the accuracy and robustness of the deployed solution. This section reviews existing datasets, specialized traffic datasets, and the need for localization to Indian driving conditions, culminating in the selection of the **Indian Driving Dataset (IDD)** as the most suitable resource.

General-Purpose Datasets

Several large-scale datasets have been developed to advance computer vision research:

- **COCO (Common Objects in Context):** Contains over 330,000 images with 80 object categories. It emphasizes contextual relationships between objects, making it useful for general object detection tasks.
- **Pascal VOC:** A pioneering dataset with 20 object categories, widely used for benchmarking detection and segmentation algorithms.
- **ImageNet:** Comprising over 14 million images across 20,000 categories, it has been instrumental in training deep learning models for classification and detection.

These datasets are easily accessible through open repositories and have been extensively used to pre-train models. However, they are **general-purpose** and lack the specificity required for traffic scenarios, especially in the Indian context.

Datasets for Traffic and Road Scenarios

To address road safety applications, several specialized datasets have been curated:

- **KITTI:** Focused on autonomous driving, it includes stereo images, optical flow, and 3D object detection data collected from European roads.
- **Cityscapes:** Provides pixel-level annotations for urban street scenes, emphasizing semantic segmentation of vehicles, pedestrians, and road infrastructure.
- **Berkeley DeepDrive (BDD100K):** A large-scale dataset with 100,000 driving videos, covering diverse weather and lighting conditions.
- **Indian Road Datasets (emerging):** Some smaller datasets exist, but they are limited in scope and scale compared to international counterparts.

While these datasets are valuable, they primarily reflect traffic conditions in developed countries, which differ significantly from Indian roads in terms of vehicle diversity, road infrastructure, and environmental factors.

Need for Localization to Indian Traffic Conditions

Indian roads present unique challenges that global datasets fail to capture:

- **Vehicle diversity:** Auto-rickshaws, two-wheelers, tractors, and overloaded trucks are common.

- **Non-standard road users:** Animals such as cows, dogs, and goats frequently appear on roads.
- **Unique road signs:** India has region-specific traffic signs and informal indicators not found in Western datasets.
- **Unstructured traffic flow:** Lane discipline is often absent, and vehicles may move unpredictably.

Without localization, models trained on foreign datasets risk poor accuracy and high false positives when deployed in Indian scenarios.

Indian Driving Dataset (IDD)

The **Indian Driving Dataset (IDD)**, available on Kaggle, is specifically designed to capture the complexity of Indian traffic. The dataset consists of images obtained from a front facing camera attached to a car. The car was driven around Hyderabad, Bangalore cities and their outskirts. The images are mostly of 1080p resolution, but there is also some images with 720p and other resolutions. The dataset is divided into train, val and test splits as follows:

Type	Images
Full	46,588
Train	31,569
Val	10,225
Test	4,794

It contains:

- **40,000+ images** collected from Indian cities like Bangalore and Hyderabad
- **34 object categories**, including cars, buses, trucks, auto-rickshaws, pedestrians, animals, and traffic signs.
- Coverage of varied conditions such as urban, semi-urban, and rural roads.

Strategies for Dataset Labelling and Augmentation

Even with specialized datasets, augmentation is essential to improve robustness and generalization:

- **Geometric transformations:** Rotation, scaling, flipping, and cropping to simulate different viewpoints.
- **Photometric adjustments:** Varying brightness, contrast, and color balance to mimic day/night and weather conditions.
- **Synthetic occlusions:** Adding artificial obstacles to simulate real-world scenarios where objects are partially hidden.
- **Domain adaptation:** Combining IDD with global datasets (e.g., COCO, Cityscapes) to enrich object diversity while maintaining localization.

- **Generative approaches:** Using GANs or diffusion models to create synthetic traffic scenes for rare events.

These strategies will ensure that the trained model remains resilient to the unpredictability of Indian roads while meeting performance objectives of accuracy and inference speed.

3.4. Summary of Literature Review

Key insights from NPU and microcontroller studies.

Lessons from object detection algorithm comparisons.

The literature shows a clear progression toward lightweight, real-time object detection models suitable for edge deployment.

- **YOLO** stands out for speed and responsiveness, making it highly suitable for driver assistance.
- **SSD** offers a balanced approach but may underperform in detecting small hazards.
- **EfficientDet** provides strong accuracy but requires careful optimization for real-time use.
- **MobileNet-based detectors** excel in resource-constrained environments but sacrifice accuracy.

For this dissertation, **YOLO11** combined with the **STM32N6 microcontroller** provides the best balance of performance, accuracy, and feasibility for a proof-of-concept driver assistance system.

Importance of dataset relevance.

The availability of general-purpose datasets has accelerated computer vision research, but their limitations in traffic-specific contexts necessitate specialized datasets. For India, localization is critical due to unique traffic conditions. The Indian Driving Dataset (IDD) emerges as the most appropriate choice, offering comprehensive coverage of relevant categories and scenarios. Through careful augmentation and integration with NPU-enabled edge devices, this project aims to deliver a cost-effective, accurate, and real-time driver assistance solution tailored to Indian roads.

IDD's richness in categories and annotations makes it highly suitable for training models that can generalize across India's heterogeneous traffic environments. Its availability on Kaggle ensures ease of access and integration into research workflows.

How these findings guide the design choices for the dissertation project.

4. Methodology & Experimental Setup

4.1. Methodology

In this section we briefly describe the methodology adopted for the experiment and setup required for the same in terms of software, hardware and data. There are three key phases of the methodology as explained below.

1. Model Training on Custom Dataset

- Prepare a custom dataset combining IDD and additional samples to reflect Indian traffic conditions.
- Pre-process images in terms of size, orientation, clarity as required for training.
- Annotate images for training.
- Train the pre-build models (YOLO11n and YOLO26n) using transfer learning.
- Evaluate the performance using mAP and precision-recall metrics.
- Make changes to image dataset, annotations, model hyper parameters and repeat training to get acceptable performance.

2. Implementation on Target System (STM32N6)

- Quantize and convert the trained model for STM32N6 deployment using STM32Cube.AI.
- Use various optimization techniques such as pruning and quantization to reduce latency and memory usage.
- Test laboratory and real-time inference and measure performance in terms of frame rate, resource utilization, and power consumption.

3. Results and Discussion

Discuss the performance of various optimization techniques.

4.2. Experimental Setup

Software Requirements

Google Collab	Model training and evaluation
Ultralytics Models	Pretrained models YOLO11n and YOLO26n
Python	Programming
Anaconda	Desktop IDE for Python programming
Label-studio	Data annotation
STM32CubeIDE	Microcontroller programming IDE
STM32CubeMX	Microcontroller configuration IDE
STM32CubeProgrammer	Microcontroller deployment IDE
STM32CubeMonitor	Microcontroller monitoring IDE
STMicroelectronics Model Zoo Services	Model conversion, quantization and deployment on Microcontroller

STMicroelectronics Model Zoo	Pretrained models for microcontrollers
---	--

Hardware Requirements

STM32N6570-DK	The STM32N6570-DK Discovery kit is a complete demonstration and development platform for the Arm® Cortex®-M55 core-based STM32N657X0H3Q microcontroller. https://www.st.com/en/evaluation-tools/stm32n6570-dk.html
----------------------	--



5. Model Training on custom dataset

The proposed driver assistance solution relies on robust object detection tailored to Indian road conditions. To achieve this, a **custom dataset** was curated by combining publicly available sources such as the **Indian Driving Dataset (IDD)** with additional annotated samples representing diverse traffic scenarios (e.g., mixed traffic, pedestrians, two-wheelers, roadside vendors).

Preprocessing:

Images were resized and normalized to ensure compatibility with the YOLO-based architecture. Data augmentation techniques (random flips, rotations, brightness adjustments) were applied to improve generalization across varying lighting and weather conditions.

Source Data set	https://www.kaggle.com/datasets/redzapdos123/indian-driving-dataset-detections-yolov11
No of images in data set	10000+
Image size	1280 x 720 pixel
No of images picked for training	618
No of images picked for validation	60
Object detection classes	15
	- autorickshaw - bicycle - bus - car - caravan - motorcycle - person - rider - traffic_light - traffic_sign (only speed limits) - trailer - train - truck - vehicle_fallback

Data Annotations:

To prepare the dataset for training, image data (wherever not consistent) was annotated using Label Studio, an open-source tool that supports customizable labelling workflows. Each image was manually annotated to mark relevant objects such as pedestrians, vehicles, and traffic signs, ensuring class consistency across the dataset. The tool's intuitive interface allowed bounding box creation and class assignment, while its project management features facilitated quality control and iterative refinement. The resulting annotations were exported in YOLO-compatible format, enabling seamless integration into the training pipeline.

Folder and file structure:

Store the images and annotation files as per below structure for training.

```
test
├── images
└── labels

train
├── images
└── labels

val
├── images
└── labels
```

Training Procedure:

The YOLOv11 and YOLOv26 framework was employed, leveraging transfer learning from pretrained weights on COCO. Hyperparameters such as learning rate, batch size, and number of epochs were tuned iteratively to balance accuracy and computational efficiency.

Training Environment	Google Collab
Software and Libraries	Google Collab Notebook Ultralytics 8.4.22 Python 3.12.2 Torch 2.10.0 Cuda 128
Hardware	1 CPU + 1 NVIDIA Tesla T4 GPU + 16 GB RAM
Training parameters	Batch = 16 Epochs = 40 Amp = True Data file = data.yaml Imgsize = 640 Model = yolo11n / yolo26n Other parameters were left at default value
Pre-trained models	Ultralytics - PyTorch - YOLO11n / YOLO26n

Evaluation Metrics:

Performance was assessed using **mean Average Precision (mAP)**, **precision-recall curves**, and **confusion matrices** to validate detection accuracy across multiple object classes relevant to Indian roads.

Output – YOLO11n

```
Validating /content/runs/detect/train/weights/best.pt...
Ultralytics 8.4.22 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,585,077 parameters, 0 gradients, 6.3 GFLOPs
```

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	61	2101	0.548	0.303	0.343	0.217
autorickshaw	52	138	0.776	0.457	0.546	0.401
bicycle	4	5	0.296	0.4	0.311	0.198
bus	37	78	0.661	0.499	0.544	0.392
car	57	469	0.68	0.591	0.653	0.431
motorcycle	57	316	0.705	0.418	0.482	0.271
person	49	279	0.495	0.262	0.28	0.14
rider	59	468	0.682	0.295	0.37	0.199
traffic_light	18	51	0.52	0.107	0.154	0.0788
traffic_sign	35	63	0.582	0.243	0.308	0.184
train	1	1	0	0	0	0
truck	38	82	0.58	0.354	0.437	0.295
vehicle_fallback	51	151	0.599	0.0106	0.0278	0.0116

Speed: 0.2ms preprocess, 2.7ms inference, 0.0ms loss, 3.5ms postprocess per image
Results saved to /content/runs/detect/train

Figure 5 - Yolo11n Performance Score

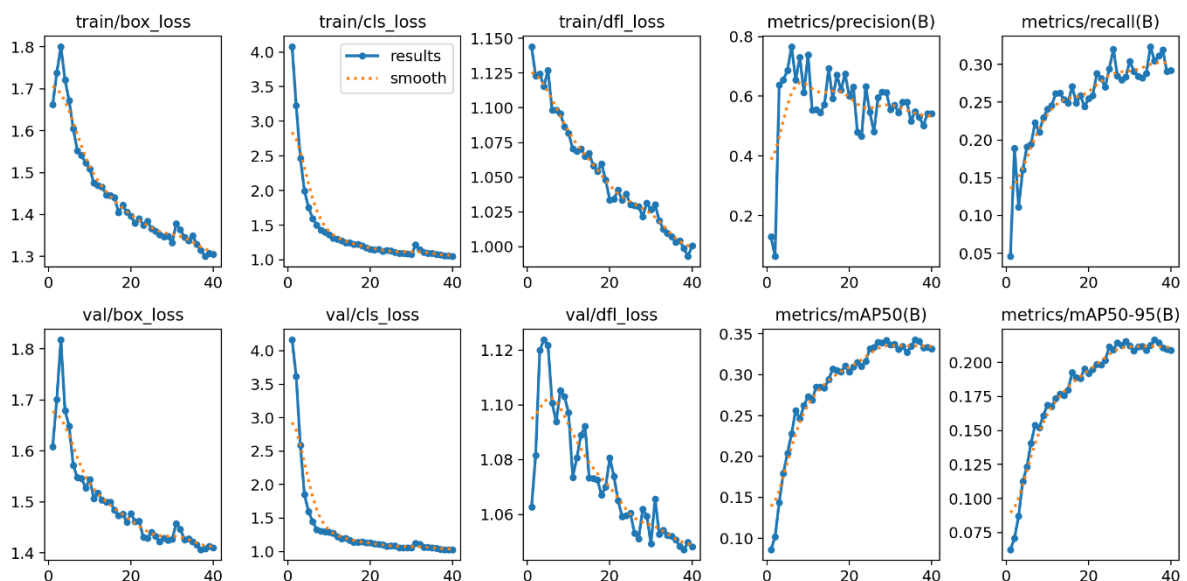


Figure 6 - Yolo11n Performance Charts

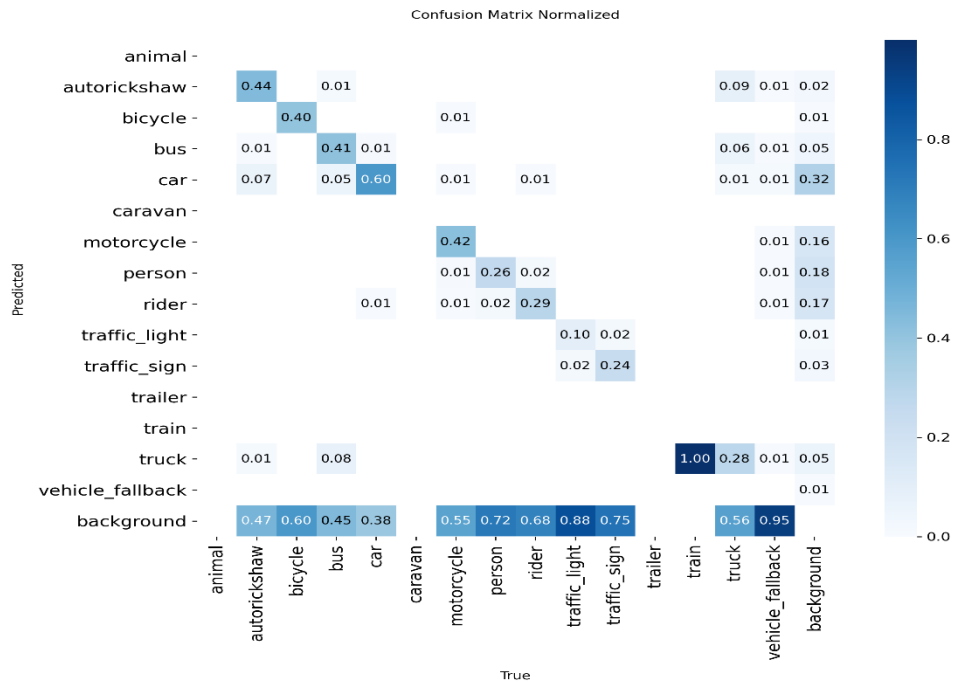


Figure 7 - Yolo11n Confusion Matrix

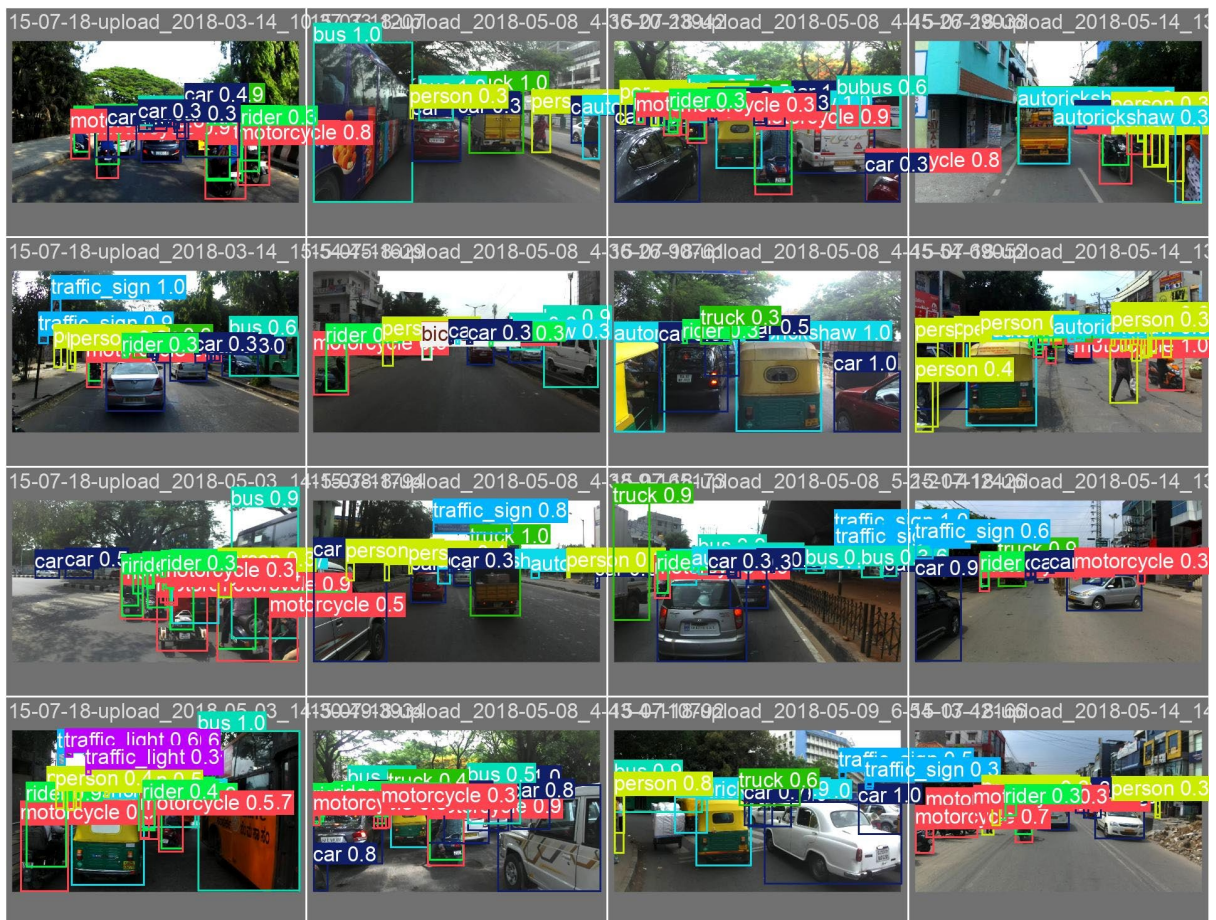


Figure 8 - Yolo11n Test Result

Output – YOLO26n

```
Validating /content/runs/detect/train2/weights/best.pt...
Ultralytics 8.4.22 🚀 Python-3.12.12 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26n summary (fused): 122 layers, 2,377,761 parameters, 0 gradients, 5.2 GFLOPs
```

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	61	2101	0.479	0.262	0.293	0.181
autorickshaw	52	138	0.57	0.435	0.478	0.343
bicycle	4	5	0.808	0.2	0.24	0.146
bus	37	78	0.568	0.474	0.53	0.342
car	57	469	0.541	0.569	0.585	0.377
motorcycle	57	316	0.529	0.421	0.429	0.231
person	49	279	0.508	0.211	0.234	0.118
rider	59	468	0.484	0.297	0.311	0.162
traffic light	18	51	0	0	0.0501	0.0287
traffic sign	35	63	0.567	0.238	0.257	0.168
train	1	1	0	0	0	0
truck	38	82	0.55	0.293	0.377	0.253
vehicle fallback	51	151	0.621	0.00662	0.0287	0.00935

Speed: 0.2ms preprocess, 2.9ms inference, 0.0ms loss, 0.2ms postprocess per image
 Results saved to /content/runs/detect/train2

Figure 9 - Yolo26n Performance Score

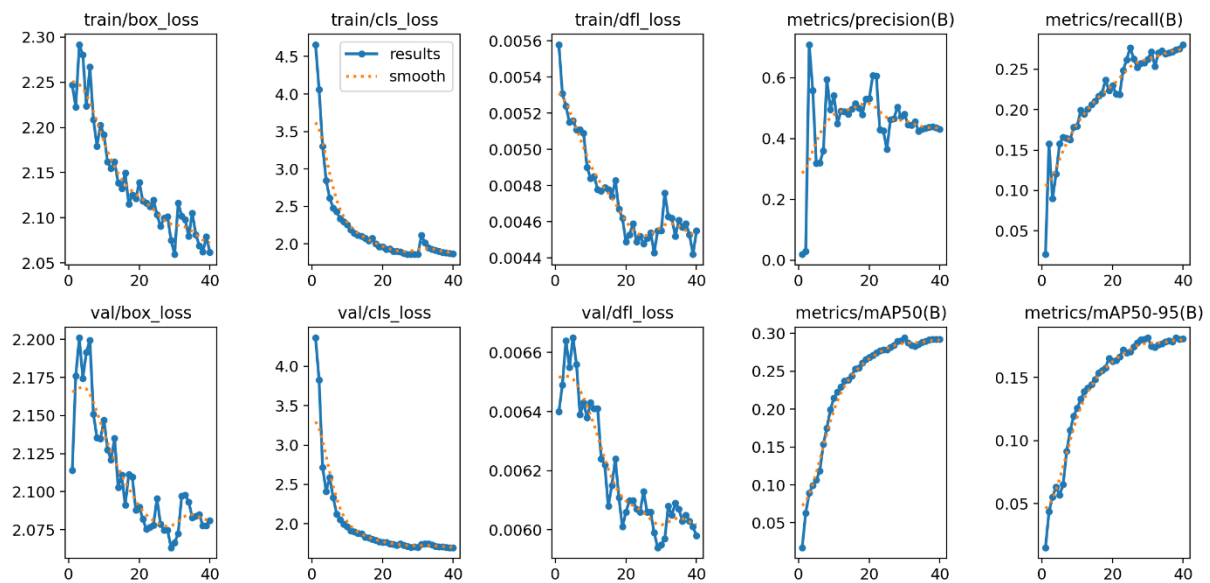


Figure 10 - Yolo26n Performance Charts

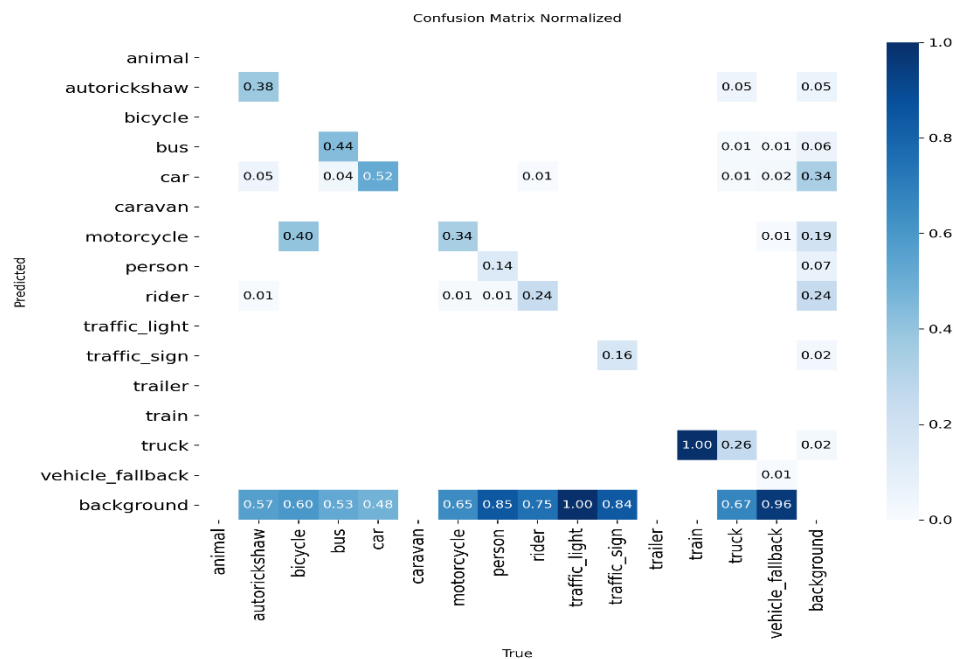


Figure 11 - Yolo26n Confusion Matrix



Figure 12 - Yolo26n Test Result

Performance Comparison (in terms of accuracy score)

Object Class	mAP50 – YOLO11n	mAP50 – YOLO26n
autorickshaw	0.546	0.478
bicycle	0.311	0.24
bus	0.544	0.53

car	0.653	0.585
motorcycle	0.482	0.429
person	0.28	0.234
rider	0.37	0.311
traffic_light	0.154	0.05
traffic_sign (only speed limits)	0.308	0.257
truck	0.437	0.377
Overall	0.343	0.293

Inference:

As seen from the above comparison we can make the following inferences.

Observation	Improvement
YOLO11n has better total accuracy compared to YOLO26n. However total mAP of both the models is around 30%.	As an improvement, need to train the model with more number and variety of images. Also, instead of image resolution of 640 pixel, we can try with original resolution of 1200 pixels.
Few object classes such as person, traffic light and traffic sign have low mAP. The possible reason could be a smaller number of training pictures.	As an improvement, need to include more pictures of these classes and ensure that their annotations are correct.

6. Implementation & Analysis (Work in progress)

- Model Conversion (e.g., ONNX to NPU-specific runtime).
- Post-Training Quantization (PTQ) strategies.
- **Layer-wise Profiling:** Which layers are the bottlenecks on the NPU? (e.g., "Custom layers often fallback to CPU").
- **Comparative Performance:** which model / optimization technique provides better results
- **Hardware Utilization:** Analyzing how effectively the CNN utilizes the NPU's specialized compute units.
- **Evaluation Metrics:** Mean Average Precision (mAP), Latency (ms), Throughput (FPS), and Energy Efficiency (Frames/Joule).

7. Results and Discussion (Work in progress)

- **The Accuracy-Latency Trade-off:**
- **Impact of Input Resolution:**
- **Generalization:**
- **Quantitative Results:**
- **Qualitative Observations:**
- **Discussion:**

8. Conclusion & Future Work (Work in progress)

- **Summary of Findings:** Which NPU architecture/CNN pairing was most effective?
- **Recommendations:** Best practices for developers deploying at the edge.
- **Limitations:** E.g., lack of support for certain operations in current NPU compilers.
- **Future Directions:** Mentioning Transformer-based models (ViTs) on NPUs.

9. References & Appendices (Work in progress)

- Literature references
- Code snippets, hardware configuration files, and detailed data tables.
- Specifications
- Architecture / Components
- Tool chain